

Лабораторна робота №1

Тема: Git та GitHub. Linux, Command Line. Git collaboration.

Мета: Ознайомитись з основними концепціями та інструментами систем контролю версій Git та платформи GitHub, а також навчитися ефективно використовувати їх для керування кодом та спільної розробки.

Вимоги до звіту:

Скріншоти повинні бути читабельними та містити достатньо інформації для підтвердження виконання відповідного завдання.

Для команд Git на скріншотах повинні бути видимими:

- команда, яку було виконано;
- результат її виконання;
- за необхідності — поточна гілка та стан репозиторію.

Не допускається використання скріншотів, на яких неможливо встановити факт виконання завдання.

Практична частина

Завдання 1 (GitHub).

1. Зареєструвати акаунт на <https://github.com/> (можете використати корпоративну пошту ЧНУ).

Завдання 2 (Git basics, Working with multiple repositories).

1. Пройдіть курс <https://githowto.com/>.
2. Продемонструвати результати виконання завдань зазначених у Part I: Git basics та Part II: Working with multiple repositories.

Завдання 3 (Learn Git Branching).

1. Виконати всі завдання у візуальному інтерактивному курсі <https://learngitbranching.js.org/>.
2. Результати виконання подати у вигляді скріншотів завершених завдань.

Завдання 4 (Linux CLI).

1. Пройдіть всі модулі (4 modules) <https://linuxsurvival.com/>.
2. Результати виконання подати у вигляді скріншотів завершених модулів.

Примітка: даний курс <https://linuxsurvival.com/> забезпечує віртуальне середовище для введення команд. Однак, якщо ви хочете пізніше використовувати ці команди на практиці, вам потрібно буде працювати з Linux CLI, оскільки багато команд не підтримуються Windows cmd або PowerShell. Ви повинні вже встановити git з <https://git-scm.com/downloads>. Він пропонує Git Bash, який можна використовувати для введення команд bash.

Завдання 5 (Connecting to GitHub with SSH, Markdown language).

1. Виконати налаштування SSH connection згідно документації [Connecting to GitHub with SSH](#) та продемонструвати роботу SSH connection.
2. Створіть публічний репозиторій та назвіть його web-technologies.
3. Ознайомтесь з налаштуваннями репозиторію у розділі Settings.
4. Клонуйте ваш репозиторій на свій комп'ютер з використанням SSH connection та команди git clone.
5. Створіть пустий файл [README.md](#) у корені вашого репозиторію web-technologies.
6. Заповніть файл README.md, додавши опис репозиторію, ПІБ студента, інформацію про улюблені мови програмування, технології тощо, використовуючи [Markdown language](#).
7. Виконайте commit для файлу README.md дотримуючись правил [Conventional Commits](#).
8. Запустіть зміни у ваш віддалений репозиторій web-technologies.
9. У локальному репозиторію створіть директорію з назвою lab-01.
10. У директорії lab-01 створіть директорію learn-git та додайте результати виконання (скріншоти) Завдання 3. Виконайте відповідний commit дотримуючись правил [Conventional Commits](#) та запусіть зміни у ваш віддалений репозиторій web-technologies.
11. У директорії lab-01 створіть директорію linux-survival, та додайте результати виконання (скріншоти) Завдання 4. Виконайте відповідний commit дотримуючись правил [Conventional Commits](#) та запусіть зміни у ваш віддалений репозиторій web-technologies.
12. Створіть у корені репозиторію файл .gitignore. Додайте до нього типові файли та директорії, які не повинні потрапляти до Git-репозиторію, наприклад node_modules/.

Завдання 6 (GitHub Collaboration: Fork, Branch, Merge та Pull Request).

1. Форкніть репозиторій <https://github.com/Computer-Science-Department-ChNU/git-collaboration> через GitHub web-interface. У результаті у вашому GitHub-акаунті має

з'явитися власна копія репозиторію (fork).

2. Клонуйте свій форк (fork) локально на свій комп'ютер з використанням SSH connection та команди `git clone`.
3. Додайте репозиторій <https://github.com/Computer-Science-Department-ChNU/git-collaboration> як upstream: `git remote add upstream git@github.com:Computer-Science-Department-ChNU/git-collaboration.git`.
4. Виконайте команду `git remote -v` та проаналізуйте те що ви побачите у терміналі. У результаті мають бути доступні щонайменше два віддалені репозиторії: origin - ваш fork, та upstream - оригінальний репозиторій.
5. Перейдіть до гілки main (`git checkout main` або `git switch main`) і потім створіть нову гілку, ім'я на ваш роздум (aka feature branch): `git checkout -b feature/student-surname`, де student-surname - прізвище студента вказується англійською мовою.
6. Внесіть деякі зміни до свого локального сховища. Для цього відкрийте файл README.md та додайте: ваше ПІБ, короткий опис того, що ви дізналися під час виконання лабораторної роботи, короткий опис ваших вражень від роботи з Git та GitHub, використовуючи [Markdown language](#). Не змінюйте та не видаляйте інформацію, додану іншими студентами.
7. Внесіть зміни до новоствореної гілки (`git commit -m "Your commit message"`). Повідомлення коміту повинно коротко описувати виконану зміну.
8. Перейдіть до гілки main: `git checkout main` або `git switch main`.
9. Витягніть останні зміни з гілки upstream main: `git pull upstream main`.
10. Об'єднайте головну гілку зі своєю гілкою: `git checkout feature/student-surname && git merge main`.
11. Вирішіть будь-які конфлікти мержу, якщо такі є (Resolve merge conflicts). Не змінюйте та не видаляйте інформацію, додану іншими студентами.
12. Надішліть гілку до вашого віддаленого сховища: `git push --set-upstream origin feature/student-surname`. Після цього ваша гілка повинна з'явитися у вашому fork на GitHub.
13. Зробіть pull-request з вашого репозиторію до репозиторію <https://github.com/Computer-Science-Department-ChNU/git-collaboration> через GitHub web-interface.
14. Pull Request повинен мати: змістовну назву, опис виконаних змін, інформацію про те, що завдання лабораторної роботи виконано.
15. Перевірте створений Pull Request та переконайтеся, що GitHub не повідомляє про конфлікти злиття.
16. Якщо після створення Pull Request виникне потреба у виправленні конфліктів або внесенні додаткових змін, виконайте необхідні зміни у своїй локальній feature-гілці та повторно виконайте команду `git push`. Нові зміни автоматично з'являться у створеному Pull Request.
17. Під час виконання роботи звертайте увагу на призначення понять fork, clone, origin, upstream, branch, commit, merge та pull request, оскільки основною метою роботи є практичне ознайомлення з моделлю спільної розробки програмного забезпечення через

Git та GitHub.

18. Зауважте, що ваш PR можуть не розглянути швидко.

У результаті виконання завдання 6 студент повинен мати:

1. fork репозиторію git-collaboration у власному GitHub-акаунті;
2. локальну копію репозиторію;
3. налаштовані віддалені репозиторії origin та upstream;
4. окрему feature-гілку;
5. щонайменше один власний коміт;
6. feature-гілку, синхронізовану з актуальною гілкою upstream/main;
7. feature-гілку, опубліковану у власному fork;
8. створений Pull Request до оригінального репозиторію.

Завдання 7 (Managing your profile README).

1. додати файл README.md до свого профілю GitHub, щоб розповісти про себе як розробника згідно з інструкцією [Managing your profile README](#).
2. Можете використати генератор профайлу [GitHub Profile README Generator](#).
3. Ведіть свій GitHub. Під час вивчення нових технологій не забувайте публікувати прогрес. Будь-які пет-проекти також потрібно зберігати. Окрім того, що з часом ви зможете дивитись на свій старий код і думати, чому він так жахливо написаний, згадки та використання технологій згаданих в резюме допоможе інтерв'юеру скласти про вас думку.

Додаткові матеріали.

Вивчіть додаткові матеріали нижче, щоб покращити свої навички. Якщо ви вважаєте, що це вплине на вашу загальну ефективність курсу, подумайте над тим, щоб повернутись до них пізніше, наприклад коли ви виконаєте всі обов'язкові завдання:

1. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/about-merge-conflicts>
2. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-using-the-command-line>
3. <https://ohshitgit.com/>
4. <https://github.com/k88hudson/git-flight-rules>

Контрольні запитання:

1. Що таке система контролю версій і які проблеми розробки програмного забезпечення вона вирішує?
2. У чому відмінність Git від GitHub?
3. Чим розподілена система контролю версій відрізняється від централізованої?
4. Що таке Git-репозиторій?
5. Що таке робоча директорія (working tree), staging area та Git repository?
6. Що таке commit і яку інформацію він містить?
7. Для чого використовується команда `git status`?
8. Як переглянути історію комітів у Git?
9. Для чого використовується команда `git diff`?
10. Що таке branch і навіщо використовувати окремі гілки під час розробки?
11. Як створити нову гілку та перемкнутися на неї?
12. У чому різниця між `git checkout` та `git switch`?
13. Що таке merge і для чого він використовується?
14. Що таке merge conflict? У яких випадках він виникає?
15. Як вирішити конфлікт злиття у локальному репозиторії?
16. Що таке branch і навіщо використовувати окремі гілки під час розробки?
17. Як створити нову гілку та перемкнутися на неї?
18. У чому різниця між `git checkout` та `git switch`?
19. Що таке merge і для чого він використовується?
20. Що таке merge conflict? У яких випадках він виникає?
21. Як вирішити конфлікт злиття у локальному репозиторії?
22. Що таке SSH і для чого він використовується під час роботи з GitHub?
23. Яка різниця між public key та private key у SSH?
24. Чому приватний SSH-ключ не можна передавати іншим особам?
25. Що таке Markdown і для чого він використовується на GitHub?